

Diseño e implementación de una calculadora de programación lineal mediante los métodos gráfico y Simplex

Working Paper - Primera entrega del Laboratorio de Optimización

Santiago Andrés Walteros Corredor

Código estudiantil 160005242 | Python | 4 de septiembre de 2026

Resumen - Se presenta el diseño y la implementación incremental de una aplicación de escritorio para resolver modelos de programación lineal con dos variables. La herramienta incorpora el método gráfico para maximización y minimización, y el método Simplex básico para maximización con restricciones de tipo menor o igual y términos independientes no negativos. La solución se desarrolló en Python con Tkinter y Matplotlib. El programa expone vértices, evaluaciones, variables entrantes y salientes, pivotes y tablas, en lugar de ocultar el procedimiento. La versión v0.5 se verificó desde la interfaz con cuatro ejercicios gráficos y tres de Simplex. Los siete resultados coincidieron con la solución manual dentro del alcance académico de la primera entrega.

Palabras clave - programación lineal, método gráfico, método Simplex, optimización, Python, Tkinter.

I. INTRODUCCIÓN

La programación lineal permite representar decisiones mediante variables, una función objetivo lineal y un conjunto de restricciones. Su utilidad académica no se limita a obtener un valor óptimo: también permite comprender cómo los recursos disponibles delimitan una región de soluciones y por qué determinados puntos son mejores que otros.

En el aprendizaje de optimización resulta importante observar el procedimiento. Por esa razón, este proyecto evita utilizar una función que resuelva el modelo de forma automática y oculta. En su lugar, implementa directamente el cálculo de intersecciones, la comprobación de factibilidad y las operaciones elementales de pivoteo.

El objetivo fue construir una calculadora sencilla, legible y explicable. La primera entrega trabaja exclusivamente con X_1 y X_2 ; resuelve gráficamente maximización y minimización, y aplica Simplex a modelos básicos de maximización. La interfaz, los algoritmos y la evidencia permanecen integrados en una sola aplicación de escritorio.

II. FUNDAMENTOS TEÓRICOS

A. Programación lineal

Un modelo de programación lineal busca maximizar o minimizar una función lineal sujeta a restricciones también lineales. Para dos variables, la forma general utilizada en este trabajo se expresa como:

$$\text{Optimizar } Z = c_1X_1 + c_2X_2$$

$$a_{i1}X_1 + a_{i2}X_2 \leq o \geq b_i, \quad X_1, X_2 \geq 0$$

Las variables representan las decisiones; los coeficientes c_1 y c_2 cuantifican su contribución al objetivo, y cada restricción limita las combinaciones permitidas. El conjunto de puntos que satisface simultáneamente todas las restricciones constituye la región factible [2].

B. Método gráfico

Con dos variables, cada restricción puede representarse mediante una recta y el semiplano que cumple la desigualdad.

La intersección de esos semiplanos produce la región factible. En un problema lineal acotado, la evaluación de la función objetivo en los puntos extremos permite identificar el máximo o el mínimo. El procedimiento implementado genera intersecciones entre restricciones y ejes, elimina duplicados, filtra por factibilidad y compara los valores de Z .

C. Método Simplex

El método Simplex recorre soluciones básicas factibles asociadas con vértices adyacentes de la región factible hasta cumplir una condición de optimalidad [3]. Para el alcance definido, cada restricción $\leq$ recibe una variable de holgura y la función objetivo se escribe como $Z - c_1X_1 - c_2X_2 = 0$.

En cada iteración se elige como variable entrante la columna con el coeficiente más negativo en la fila Z . Las razones entre la columna de resultados y los coeficientes positivos determinan la variable saliente. Después se normaliza la fila pivote y se hacen ceros en el resto de la columna. Cuando la fila Z no contiene coeficientes negativos relevantes, la tabla se considera óptima.

III. ANÁLISIS Y DISEÑO

A. Alcance e historias de usuario

El análisis produjo diez historias de usuario. Para mantener el informe compacto, la Tabla I las agrupa según la tarea que representan. Todas se reflejan en los controles y salidas de la versión v0.5.

TABLA I. RESUMEN DE HISTORIAS DE USUARIO

HU	Necesidad principal	Respuesta implementada
01-05	Configurar y adaptar el modelo	Método, objetivo, coeficientes y restricciones dinámicas
06-07	Resolver por método gráfico	Vértices, evaluaciones, región y óptimo
08-10	Resolver Simplex y reiniciar	Tablas, pivotes, iteraciones y limpieza

El método gráfico acepta restricciones $\leq$ y $\geq$, mientras que Simplex v1.0 se limita a maximización, restricciones $\leq$ y valores b no negativos. Se excluyeron igualdades, variables artificiales, Gran M, dos fases y más de dos variables.

B. Arquitectura

Aunque la entrega conserva un único archivo Python, las responsabilidades se separaron en cuatro clases. ProblemaPL almacena el modelo; MetodoGrafico y MetodoSimplex ejecutan los algoritmos; AplicacionPL administra Tkinter y presenta la información. Esta separación evita que los cálculos dependan de los controles visuales.

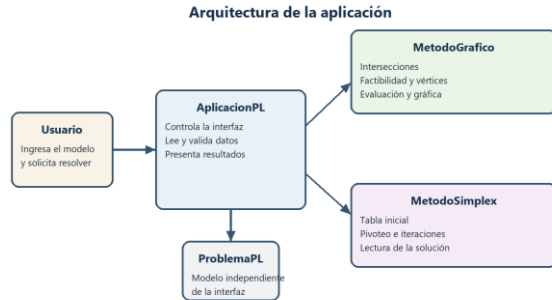


Fig. 1. Arquitectura lógica de la aplicación.

C. Flujo de resolución

La interfaz convierte el formulario en un objeto de problema, comprueba las condiciones del método seleccionado y delega el cálculo. Ambos algoritmos devuelven estructuras de datos que la interfaz transforma en texto, tablas o gráfica.

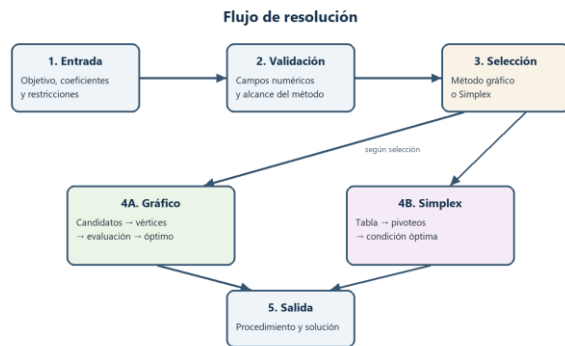


Fig. 2. Flujo común y ramas de los métodos.

IV. IMPLEMENTACIÓN

A. Tecnologías y estructura

La aplicación se desarrolló en Python. Tkinter y ttk suministran la ventana, los campos, las listas desplegables y los paneles; la documentación oficial describe Tkinter como la interfaz de Python para Tcl/Tk [4]. Matplotlib se incrusta mediante FigureCanvasTkAgg, siguiendo el mecanismo oficial para integrar figuras en Tk [5]. No se utilizó SciPy como motor de optimización.

El desarrollo avanzó por versiones verificables: v0.1 definió alcance y diseño; v0.2 construyó la interfaz y la captura del modelo; v0.3 completó el método gráfico; v0.4 implementó Simplex; y v0.5 integró ambos métodos y las pruebas manuales.

B. Método gráfico implementado

Las fronteras se procesan de dos en dos mediante determinantes. También se calculan los cortes con X1 y X2 y se agrega el origen como candidato. Cada punto se compara con todas las desigualdades usando una tolerancia numérica; solo los puntos

no negativos y factibles permanecen como vértices. Finalmente se evalúa Z y se selecciona el extremo solicitado.

Para dibujar la región, los vértices se ordenan por el ángulo respecto de su centro. La figura presenta las rectas, el polígono factible, los vértices numerados y una estrella roja en la solución óptima.

C. Simplex implementado

La tabla inicial contiene las columnas X1, X2, una holgura por restricción y CR. Las variables de holgura forman la base inicial. Cada iteración conserva una copia de la tabla, la variable entrante, las razones, la variable saliente y el elemento pivote, de modo que el usuario puede reconstruir el procedimiento mostrado.

La solución se lee desde la columna CR: una variable de decisión vale el resultado de su fila cuando pertenece a la base y vale cero en caso contrario. El valor de Z se toma de la última fila. Los errores pequeños de punto flotante se normalizan para presentar enteros cuando corresponde.

D. Interfaz integrada

El formulario inicia con dos restricciones y permite agregar o retirar filas. Cuando se selecciona Simplex, el objetivo se fija en maximizar y los operadores se bloquean en $\leq$. Al modificar cualquier dato se descarta el resultado anterior, lo que evita presentar una solución que ya no corresponde con el modelo visible.

Fig. 3. Panel para configurar el modelo.

V. RESULTADOS

La versión v0.5 se ejecutó desde la interfaz con siete modelos conocidos. Los valores se compararon con la solución manual usando una tolerancia de 0.000001. Las cuatro pruebas gráficas y las tres pruebas Simplex coincidieron.

A. Pruebas del método gráfico

TABLA II. RESULTADOS DEL MÉTODO GRÁFICO

Caso	Objetivo	Manual	Programa
G-01	Máx.	(30,40); 2900	(30,40); 2900

Caso	Objetivo	Manual	Programa
G-02	Máx.	(50,50); 4000	(50,50); 4000
G-03	Mín.	(4,0); 8	(4,0); 8
G-04	Mín.	(2,3); 5	(2,3); 5

En G-01 se resolvió $\text{Max } Z = 30X_1 + 50X_2$ con $X_1 + 3X_2 \leq 150$ y $X_1 + X_2 \leq 70$. Se obtuvieron cuatro vértices factibles y el punto (30, 40), con $Z = 2900$, fue identificado como óptimo.

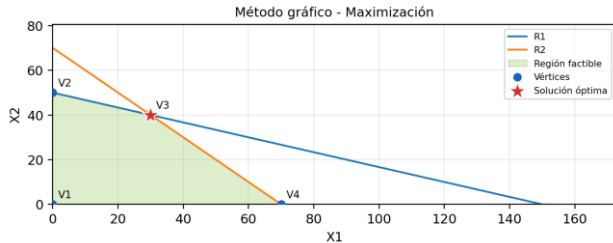


Fig. 4. Región factible y solución óptima del caso G-01.

B. Pruebas del método Simplex

TABLA III. RESULTADOS DEL MÉTODO SIMPLEX

Caso	Iter.	Manual	Programa
S-01	2	(30,40); 2900	(30,40); 2900
S-02	2	(1,3); 9	(1,3); 9
S-03	2	(2,6); 36	(2,6); 36

El caso S-01 usa el mismo modelo que G-01. La tabla inicial incorpora S1 y S2; después de dos pivoteos, X2 y X1 forman la base y la fila Z registra 2900. Las Figuras 5 y 6 muestran la evidencia inicial y final generada por la aplicación.

Procedimiento y resultados

Método seleccionado: Método Simplex

Modelo ingresado

Max $Z = 30X_1 + 50X_2$

s.a.

$1X_1 + 3X_2 \leq 150$

$1X_1 + 1X_2 \leq 70$

$X_1, X_2 \geq 0$

Tabla inicial

VB	X1	X2	S1	S2	CR
S1	1	3	1	0	150
S2	1	1	0	1	70

Fig. 5. Tabla inicial del caso S-01.

Procedimiento y resultados

X2: 150

S2: 30

Variable saliente: S2

Elemento pivote: 0.6666666667

Tabla de la iteración 2

VB	X1	X2	S1	S2	CR
X2	0	1	0.5	-0.5	40
X1	1	0	-0.5	1.5	30
Z	0	0	10	20	2900

Solución final

X1 = 30

X2 = 40

Z = 2900

Fig. 6. Segunda iteración y solución final de S-01.

C. Interpretación

La coincidencia de 7 de 7 casos demuestra que la implementación reproduce los procedimientos esperados dentro del alcance evaluado. La prueba G-02 confirmó el manejo de cotas inferiores en el método gráfico; G-03 y G-04 comprobaron minimización con restricciones $\geq$; y S-03 verificó la construcción de tres variables de holgura.

VI. LIMITACIONES

Los resultados no deben generalizarse a casos excluidos. Simplex v1.0 no acepta minimización, restricciones $\geq$ o $=$, términos independientes negativos, variables artificiales, Gran M ni dos fases. Tampoco detecta explícitamente degeneración, soluciones múltiples, infactibilidad o problemas no acotados. El método gráfico se limita a dos variables y supone una región poligonal adecuada para los casos académicos utilizados.

Estas restricciones fueron decisiones conscientes para priorizar una primera versión comprensible. La segunda entrega ampliará Simplex a minimización sin presentar como implementadas funcionalidades que todavía pertenecen al trabajo futuro.

VII. CONCLUSIONES

El proyecto permitió relacionar el modelo algebraico con su interpretación geométrica. La construcción de intersecciones y el filtrado de puntos mostraron por qué la solución óptima se busca entre los vértices factibles, mientras que la gráfica hizo visible el efecto conjunto de las restricciones.

La implementación de Simplex evidenció que el pivoteo no es una operación aislada: cada elección de variable entrante y saliente cambia la base y conduce a una nueva solución factible. Conservar las tablas y razones dentro de la interfaz facilitó seguir el proceso hasta la condición de optimalidad.

El desarrollo incremental redujo el riesgo de mezclar errores de interfaz, modelado y cálculo. Verificar cada versión antes de integrar permitió alcanzar una v0.5 en la que ambos métodos comparten el mismo formulario y los siete casos de prueba coinciden con las soluciones manuales. La aplicación cumple el propósito de ser funcional, sencilla y explicable, dentro de las limitaciones declaradas.

REFERENCIAS

- [1] S. A. Walteros Corredor, "Actividad N.º 2 - Taller Programación Lineal", material de la asignatura, 2026.
- [2] NEOS Guide, "Linear Programming", University of Wisconsin-Madison. Disponible: <https://neos-guide.org/guide/types/lp/> (consulta: 31-08-2026).
- [3] NEOS Guide, "Simplex Method", University of Wisconsin-Madison. Disponible: <https://neos-guide.org/guide/algorithms/simplex/> (consulta: 31-08-2026).
- [4] Python Software Foundation, "tkinter - Python interface to Tcl/Tk", Python Documentation. Disponible: <https://docs.python.org/3/library/tkinter.html> (consulta: 31-08-2026).
- [5] Matplotlib Development Team, "Embedding in Tk", Matplotlib Documentation. Disponible: https://matplotlib.org/stable/gallery/user_interfaces/embedding_in_tk_sgskip.html (consulta: 31-08-2026).